

Tema 5.2

Funciones

- 1) [¿Cómo se usan las funciones?](#)
- 2) [Funciones numéricas](#)
- 3) [Funciones con cadenas](#)
- 4) [Funciones de trabajo con nulos](#)
- 5) [Funciones de manejo de fechas e intervalos](#)
- 6) [Funciones de conversión](#)
- 7) [Funciones de control de flujo](#)
- 8) [Funciones de listas](#)
- 9) [Funciones de agrupación o de totales](#)

1. Cómo se usan las funciones

Todos los SGBD implementan funciones para facilitar la creación de consultas complejas. Las **funciones** no son más que una *extensión de los operadores y por tanto se usan también para crear expresiones y realizar cálculos a partir de los datos de la bd.* Esas funciones dependen del SGBD que utilicemos, aunque hay muchas estándar. En estos apuntes se comentan las más significativas tal como se utilizan con MariaDB.

Información completa de funciones en MariaDB se puede consultar en su documentación oficial: <https://mariadb.com/kb/en/built-in-functions/>.

Todas las funciones devuelven un resultado que procede de un determinado cálculo. La *mayoría de funciones precisan que se les envíen datos de entrada, denominados **parámetros***, que son los objetos sobre los que se realizarán los cálculos de la función. *Los argumentos pueden ser columnas, expresiones, variables o constantes.* El resultado de la función, que siempre es un valor, *lógicamente depende de los **argumentos** enviados.* Éstos se pasan entre paréntesis. Por tanto, la forma de invocar a una función es:

nombreFunción([parámetro1, parámetro2,...])

y devuelve el resultado, que es un único valor.

En realidad, hay *dos tipos de funciones*:

- *Funciones que operan con datos de la misma fila.*
- *Funciones que operan con datos de varias filas diferentes (funciones de agrupación).*

2. Funciones numéricas

Función	Descripción
ROUND (n,[decimales])	Redondea el número con los decimales indicados. Si se omiten los decimales, redondea a entero. Si los decimales son negativos, redondea a la izquierda de la coma decimal. Ejem: SELECT ROUND(8.239,2); → Devuelve 8.24 ROUND(8.239); → 8 ROUND(2348.239,-2); → 2300
TRUNCATE (n,decimal es)	En lugar de redondear, corta los decimales para que sólo quede el número de decimales indicado. En este caso no se pueden omitir los decimales, pero si se indica el valor 0 en este campo, el resultado no tiene parte fraccionaria (trunca al entero). Si los decimales son negativos, trunca a la izquierda de la coma decimal. Ejem: SELECT TRUNCATE(3.57,1) → Devuelve 3.5 TRUNCATE(3.57) → ERROR TRUNCATE(3.57, 0) → 3 TRUNCATE(392.57, -2) → 300 TRUNCATE(-392.57, -2) → -300
CEIL (n) ó CEILING (n)	Valor entero más próximo y superior a <i>n</i> Ejem: SELECT CEIL(2.7); → 3 CEIL(2) → 2 CEIL(-2.3) → -2 CEIL(-2) → -2
FLOOR (n)	Valor entero más próximo e inferior a <i>n</i> -- Parte entera Ejem: FLOOR(2.7) → 2 FLOOR(2) → 2

	FLOOR(-2.3) → -3 FLOOR(-2) → -2	
MOD (n1,n2)	Devuelve el resto del resultado de dividir <i>n1</i> entre <i>n2</i> Ejem: MOD(11,4) → 3 MOD(2,5) → 2 (2 entre 5=0 y de resto 2)	
POWER (valor,exponente) ó POW (valor,exponente)	Eleva el valor al exponente indicado Ejem: SELECT POWER(7, 2); → 49	
SQRT (n)	Calcula la raíz cuadrada de <i>n</i> (<i>n</i> no puede ser negativo) Ejem: SQRT(9) → 3, SQRT(14) → 3.74...	
SIGN (n)	Devuelve 1 si <i>n</i> es positivo, cero si vale cero y -1 si es negativo	
ABS (n)	Calcula el valor absoluto de <i>n</i> Ejem: ABS(2) → 2 ABS(-2) → 2	
EXP (n)	Calcula e aumentado al número n, donde e=2.7182818	Para aplicaciones matemáticas
LN (n)	Logaritmo natural de <i>n</i>	
LOG10 (n)	Logaritmo en base 10 de <i>n</i>	
SIN (n) COS (n) TAN (n) ACOS (n) ASIN (n) ATAN (n)	Trigonométricas	
DEGREES (n)	Convierte de radianes a grados	
RADIANS (n)	Convierte de grados a radianes	

*** EJERCICIOS – Realiza los ejercicios 1 y 2 ***

3. Funciones con cadenas

Función	Descripción
LOWER (texto) ó LCASE (texto)	Convierte el texto a minúsculas (según el conjunto de caracteres que se esté usando, por defecto <i>latin1</i>). Ejem: SELECT LOWER('JESUS'); → jesus SELECT LOWER(nombre) FROM NOMBRES; → Devuelve los nombres de la columna “nombre” en minúsculas (pedro, juan, maría...).
UPPER (texto) ó UCASE (texto)	Convierte el texto a mayúsculas Ejem: UPPER('jesus') → JESUS * UPPER y LOWER son ineficaces cuando se aplican a cadenas binarias (BINARY, VARBINARY, BLOB). Para realizar una conversión a minúsculas o mayúsculas, habría previamente que convertir la cadena a una cadena no binaria con la función CONVERT. SET @str = BINARY 'North Carolina'; SELECT LOWER(CONVERT(@str USING latin1));
RTRIM (texto)	Elimina los espacios blancos a la derecha del texto. Ejem: RTRIM('JESUS ') → JESUS
LTRIM (texto)	Idem que anterior por la izquierda Ejem: LTRIM(' JESUS') → JESUS
TRIM (texto)	Elimina los espacios en blanco a la izquierda y la derecha del texto. Ejem: SELECT TRIM(' babJESUSaa '); → babJESUSaa
TRIM ([{BOTH LEADING TRAILING}] string)	Devuelve la cadena <i>str</i> con todos los prefijos <i>remstr</i> eliminados. Si no se especifica ninguno de los especificadores <i>BOTH</i> , <i>LEADING</i> ó <i>TRAILING</i> , se

TRAILING} [remstr] FROM] str), TRIM([remstr FROM] str)	asume BOTH. <i>remstr</i> es opcional y, si no se especifica, se eliminan los espacios. Ejem: SELECT TRIM(' jesus '); → jesus SELECT TRIM(LEADING 'x' FROM 'xxxbarxxx'); → barxxx SELECT TRIM(BOTH 'x' FROM 'xxxbarxxx'); → bar SELECT TRIM(TRAILING 'xyz' FROM 'barxxyz'); → barx Sobre un campo de una tabla, ejemplo: SELECT direc, TRIM('C/' FROM direc) FROM alumnos; <table border="1"> <thead> <tr> <th>direc</th><th>TRIM('C/' FROM direc)</th></tr> </thead> <tbody> <tr> <td>C/Las Matas, 24</td><td>Las Matas, 24</td></tr> <tr> <td>C/Mina 28 - 3A</td><td>Mina 28 - 3A</td></tr> <tr> <td>C/Luis Vives 25</td><td>Luis Vives 25</td></tr> </tbody> </table>	direc	TRIM('C/' FROM direc)	C/Las Matas, 24	Las Matas, 24	C/Mina 28 - 3A	Mina 28 - 3A	C/Luis Vives 25	Luis Vives 25
direc	TRIM('C/' FROM direc)								
C/Las Matas, 24	Las Matas, 24								
C/Mina 28 - 3A	Mina 28 - 3A								
C/Luis Vives 25	Luis Vives 25								
CONCAT (c1, c2,...)	Encadena <i>c1</i> y <i>c2</i> (y el resto de argumentos, si los hay) Ejem: SELECT CONCAT('mar', 'ia') maria; → maria SELECT CONCAT(direc, ' Población: ', pobla) FROM alumnos; → C/ Las Matas, 3 Población: Madrid								
CONCAT_WS (sep, c1, c2,...)	Significa “Concatenate With Separator”, es decir, concatena con separador. El primer argumento es el separador para el resto de los argumentos. El separador puede ser una cadena, como el resto de argumentos. Ejem: SELECT CONCAT_WS('---', direc, ' Población ', pobla) FROM alumnos; → C/ Las Matas, 3 --- Población --- Madrid								
SUBSTR ó SUBSTRING Usos: SUBSTRING(str, pos) SUBSTRING(str FROM pos) SUBSTRING(str, pos, len) SUBSTRING(str FROM pos FOR len)	Sin el parámetro <i>len</i>, la función devuelve una subcadena de la cadena <i>str</i> comenzando en la posición <i>pos</i>. Con el parámetro <i>len</i>, devuelve una subcadena de <i>len</i> caracteres de la cadena <i>str</i> comenzando en la posición <i>pos</i>. Si <i>pos</i> es un valor negativo, el comienzo de la subcadena es <i>pos</i> caracteres desde el final de la cadena (en vez de desde el comienzo). Ejem: SELECT SUBSTR('Logroño',-3,2); ---> 'oñ' SUBSTR('maria', 1, 3); → mar SUBSTR('maria', 2); → aria SUBSTR('maria', -2); → ia SELECT SUBSTR(fecha_alt, 6, 2) AS Mes FROM emple; → Del campo fecha_alt (1990-01-20 0000:00), está extrayendo el mes de alta en la empresa (a partir del sexto carácter, los dos siguientes).								
LEFT (str, len) RIGHT (str, len)	Devuelve los <i>len</i> caracteres de la cadena <i>str</i>, empezando por la izquierda / derecha. Ejem: SELECT LEFT('MariaDB', 5); → Maria SELECT RIGHT('MariaDB', 2); → DB								
LENGTH (texto)	Obtiene la longitud de la cadena <i>texto</i> medida en bytes (para un carácter multi-byte, habría que multiplicar la cadena por el número de bytes de cada carácter; es decir, una cadena con 5 caracteres donde cada carácter ocupa 2 bytes, devolverá 10 (con CHAR_LENGTH devolvería 5). Si <i>texto</i> no es una cadena, el parámetro se convierte a cadena automáticamente. Ejem: SELECT LENGTH('JESUS '); → 6 SELECT LENGTH(direc) FROM alumnos; → Devuelve las longitudes de las direcciones almacenadas en alumnos.								

	SELECT LENGTH(emp_no) FROM emple; → Devuelve 4 (longitud del campo emp_no)
CHAR_LENGTH (texto)	Devuelve la longitud de la cadena <i>texto</i> , medida en caracteres, ocupen estos uno o múltiples bytes. Si <i>texto</i> no es una cadena, el parámetro se convierte a cadena automáticamente.
INSTR (str, substr)	Devuelve la posición de la primera ocurrencia de <i>substr</i> dentro de <i>str</i> . Realiza la búsqueda sin tener en cuenta las mayúsculas. Si no la encuentra, devuelve 0. Ejem: SELECT INSTR('JESUS', 'S'); → 3 SELECT INSTR(titulo, 'quijote') FROM libro; → Devolvería la posición de la cadena quijote (sin distinguir mayúsculas o no) en cada una de las filas de la tabla libro → 5, 5, 0, 0 (en la primera y segunda filas empieza en la posición 5, en la tercera y cuarta no existe).
STRCMP (expr1, expr2)	Devuelve 0 si las cadenas son iguales; -1 si el primer argumento es más pequeño que el segundo según un orden alfabético y 1 en caso contrario. Ejem: SELECT STRCMP('text', 'text2'); → -1 SELECT STRCMP('text2', 'text'); → 1 SELECT STRCMP('text', 'text'); → 0
LOCATE (substr, str) LOCATE (substr, str, pos) POSITION (substr IN str)	Sin <i>pos</i> , devuelve la posición de la primera ocurrencia de la subcadena <i>substr</i> en la cadena <i>str</i> . Con <i>pos</i> , devuelve la posición de la primera ocurrencia de la subcadena <i>substr</i> en la cadena <i>str</i> a partir de la posición <i>pos</i> . Devuelve 0 si <i>substr</i> no está en <i>str</i> . No diferencia mayúsculas y minúsculas. Ejem: SELECT LOCATE('bar', 'foobarbar'); → 4 SELECT LOCATE('My', 'Maria'); → 0 SELECT LOCATE('bar', 'foobarbar', 5); → 7
REPLACE (str, from_str, to_str)	Devuelve la cadena <i>str</i> con todas las ocurrencias de la cadena <i>from_str</i> reemplazadas por la cadena <i>to_str</i> . Esta función sí tiene en cuenta mayúsculas/minúsculas. Ejem: SELECT REPLACE('JESUS', 'S', 'B'); → JEBUB SELECT REPLACE(titulo, 'Quijote', '***') FROM libro; → Cambia los Quijote (escrito la 1ª con mayúsculas) por *** en las filas de la columna título.
REVERSE (texto)	Invierte el texto (le da la vuelta) Ejem: SELECT REVERSE ('ROMA PARIS'); → SIRAP AMOR SELECT REVERSE(editorial) FROM libro;
LPAD (str, len [,padstr]) RPAD (str, len [,padstr])	Rellena el texto <i>str</i> a la izquierda (LPAD) o a la derecha (RPAD) con el carácter indicado en <i>padstr</i> para ocupar la anchura indicada en <i>len</i> . Si el texto es más grande que la anchura indicada, el texto se recorta. Si no se indica carácter de relleno, se rellenará el espacio marcado con espacios en blanco. Ejem: SELECT LPAD('Hola',10,'-'); → Da como resultado -----Hola LPAD('LUCIA', 9, '.'); →LUCIA LPAD('LUCIA', 9); → ' LUCIA' RPAD('LUCIA', 9, '.'); → LUCIA....

	RPAD('LUCIA', 9) → 'LUCIA ' SELECT LPAD(numpag, 6, '---') FROM libro; → ---240, ---327
ASCII (carácter)	Devuelve el código ASCII del carácter indicado Ejem: SELECT ASCII('a'); → 97.
CHR (número)	Devuelve el carácter correspondiente al código ASCII indicado. Ejem: SELECT CHR(67); → C.
SOUNDEX (texto)	Devuelve el valor fonético del texto. Busca textos de los que se no se sabe con exactitud su escritura. Ejem: SELECT * FROM EMPLE WHERE SOUNDEX(apellido)=SOUNDEX('RE'); → Busca a los empleados cuyo apellido suena como RE → REY SOUNDEX('Hello'); → H400

4. Funciones de trabajo con nulos

Permiten definir valores a utilizar en el caso de que las expresiones tomen el valor nulo.

Función	Descripción
NVL (valor,sustituto)	Si el <i>valor</i> es NULL, devuelve el valor <i>sustituto</i> ; de otro modo, devuelve <i>valor</i> Ejem: <i>SELECT apellido, salario+NVL(comision,0) FROM emple;</i>
NVL2 (valor,sustituto1, sustituto2)	Variante de la anterior, devuelve el valor <i>sustituto1</i> si <i>valor</i> no es nulo. Si <i>valor</i> es nulo devuelve el <i>sustituto2</i> Ejem: <i>SELECT apellido, salario+NVL2(comision,10000,0) FROM emple;</i>
COALESCE (listaExpresiones)	Permite comprobar varios valores NULL de columnas en una misma función evitando así funciones IF-THEN. Comprueba cada columna y devuelve la primera de las expresiones que no es nula. Si todas las columnas fueran NULL, la función tomará el valor por defecto que se le pasa como último parámetro. Ejemplo: <i>CREATE TABLE test (col1 VARCHAR(1), col2 VARCHAR(1), col3 VARCHAR(1));</i> <i>INSERT INTO test VALUES (NULL, 'B', 'C');</i> <i>INSERT INTO test VALUES ('A', NULL, 'C');</i> <i>INSERT INTO test VALUES (NULL, NULL, 'C');</i> <i>INSERT INTO test VALUES ('A', 'B', 'C');</i> <i>SELECT COALESCE(col1, col2, col3) FROM test;</i> El resultado es: B A C A
NULLIF (valor1,valor2)	Devuelve nulo si <i>valor1</i> es igual a <i>valor2</i> . De otro modo devuelve <i>valor1</i> Ejem: <i>SELECT apellido, NULLIF(salario, comision) FROM emple;</i>

	<table><tr><th>apellido</th><th>NULLIF(salario, comision)</th></tr><tr><td>CACERES</td><td>2000</td></tr><tr><td>SANCHEZ</td><td>1040</td></tr><tr><td>ARROYO</td><td>1500</td></tr></table>	apellido	NULLIF(salario, comision)	CACERES	2000	SANCHEZ	1040	ARROYO	1500								
apellido	NULLIF(salario, comision)																
CACERES	2000																
SANCHEZ	1040																
ARROYO	1500																
IFNULL (expr1, expr2)	Si <i>expr1</i> no es nulo, IFNULL() devuelve <i>expr1</i>; si no, devuelve <i>expr2</i>. Ejem: SELECT apellido, salario, comision, IFNULL(comision, 0) FROM emple; <table><tr><th>apellido</th><th>salario</th><th>comision</th><th>IFNULL(comision, 0)</th></tr><tr><td>CACERES</td><td>2000</td><td>NULL</td><td>0</td></tr><tr><td>SANCHEZ</td><td>1040</td><td>NULL</td><td>0</td></tr><tr><td>ARROYO</td><td>1500</td><td>390</td><td>390</td></tr></table>	apellido	salario	comision	IFNULL(comision, 0)	CACERES	2000	NULL	0	SANCHEZ	1040	NULL	0	ARROYO	1500	390	390
apellido	salario	comision	IFNULL(comision, 0)														
CACERES	2000	NULL	0														
SANCHEZ	1040	NULL	0														
ARROYO	1500	390	390														

*** EJERCICIOS – Realiza el ejercicio 3 ***

5. Funciones de manejo de fechas e intervalos

Hay que tener en cuenta que para que los nombres de los meses y las abreviaciones aparezcan en español, hay que configurar la variable global *lc_time_names*. Esta variable afecta al resultado de las funciones *DATE_FORMAT*, *DAYNAME* y *MONTHNAME*.

Para configurar la variable global a nuestro idioma hay que hacer lo siguiente:

SET lc_time_names = 'es_ES';

Una vez hecho esto, se puede consultar su valor indicando:

SELECT @@lc_time_names;

Función	Descripción				
SYSDATE ([precisión])	Obtiene la fecha y hora actuales del servidor en el que está instalada la base de datos. <i>Precisión</i> determina la precisión en microsegundos. Ejem: SELECT SYSDATE(); <table> <tr> <td>SYSDATE()</td> <td>2021-01-16 18:06:54</td> </tr> </table> SELECT SYSDATE(4); <table> <tr> <td>SYSDATE(4)</td> <td>2021-01-16 18:09:32.6630</td> </tr> </table>	SYSDATE()	2021-01-16 18:06:54	SYSDATE(4)	2021-01-16 18:09:32.6630
SYSDATE()	2021-01-16 18:06:54				
SYSDATE(4)	2021-01-16 18:09:32.6630				
NOW ([precisión]) ó CURRENT_TIMESTAMP ([precisión])	Como SYSDATE, obtiene la fecha y hora actuales del servidor en el que está instalada la base de datos. La diferencia respecto a SYSDATE es que SYSDATE devuelve el tiempo en el cual la función se ejecuta, mientras que NOW devuelve el tiempo en el cual comenzó a ejecutarse la sentencia que incluya a dicha función. A esto se le puede ver más sentido cuando se usan estas funciones en procedimientos almacenados y en triggers. Ejem: SELECT NOW(), SLEEP(2), NOW();				

	NOW() SLEEP(2) NOW() 2021-01-16 18:12:57 0 2021-01-16 18:12:57 SELECT SYSDATE(), SLEEP(2), SYSDATE(); SYSDATE() SLEEP(2) SYSDATE() 2021-01-16 18:13:31 0 2021-01-16 18:13:33
CURRENT_DATE() ó CURDATE()	Obtiene la fecha actual en formato 'YYYY-MM-DD' ó 'YYYYMMDD' dependiendo de si la función es utilizada en un contexto de cadena o numérico. Ejem: SELECT CURDATE(); → 2021-01-16
CURRENT_TIME ([precisión]) ó CURTIME ([precisión])	Devuelve la hora actual en formato 'HH:MM:SS' ó 'HHMMSS.uuuuuu' dependiendo de si la función es utilizada en un contexto de cadena o numérico. Ejem: SELECT CURTIME(); → 18:20:51 SELECT CURTIME(3); → 18:21:20:633
DATE (expr)	Extrae la parte de la fecha de una expresión en formato fecha ó fecha/hora. EJEM: SELECT DATE('2020-12-29 12:21:32'); → 2020-12-29
TIME (expr)	Extra la parte de la hora de una expresión en formato hora ó fecha/hora. EJEM: SELECT TIME('2020-12-31 01:02:03'); → 01:02:03
DATE_ADD (date,INTERVAL expr unit) y DATE_SUB (date,INTERVAL expr unit)	Realiza operaciones aritméticas con fechas. El argumento <i>date</i> especifica la fecha (o fecha/hora) de comienzo. <i>Expr</i> es una expresión que especifica el intervalo a ser añadido o sustraído de la fecha de comienzo. <i>Expr</i> es una cadena; puede comenzar por “-“ para intervalos negativos. <i>Unit</i> es una palabra reservada que indica las unidades en la cual la expresión debería ser interpretada. Ejem: SELECT '2020-12-31 23:59:59' + INTERVAL 1 SECOND; → 2021-01-01 00:00:00 SELECT INTERVAL 1 DAY + '2020-12-31'; → 2021-01-01 SELECT '2021-01-01' - INTERVAL 1 SECOND; → 2020-12-31 23:59:59 SELECT DATE_ADD('2020-12-31 23:59:59', INTERVAL 1 SECOND); → 2021-01-01 00:00:00 SELECT DATE_ADD('2020-12-31 23:59:59', INTERVAL 1 DAY); → 2021-01-01 23:59:59 SELECT DATE_ADD('2020-12-31 23:59:59', INTERVAL '1:1' MINUTE_SECOND); → 2021-01-01 00:01:00 SELECT DATE_ADD('2021-01-01 00:00:00', INTERVAL '-1 10' DAY_HOUR); → 2020-12-30 14:00:00 SELECT DATE_SUB('2021-01-02', INTERVAL 31 DAY); → 2020-12-02 SELECT DATE_SUB('2021-01-01 00:00:00', INTERVAL '1 1:1:1' DAY_SECOND); → 2020-12-30 22:58:59
ADDDATE (date, INTERVAL expr unit) ó ADDDATE (expr, days)	Cuando se utiliza la sintaxis con <i>INTERVAL</i> en el segundo argumento, ADDDATE es sinónimo de DATE_ADD. Ejem: SELECT DATE_ADD('2021-01-02', INTERVAL 31 DAY); → 2021-02-02

	Cuando se utiliza la sintaxis con <i>days</i>, este argumento referencia el número de días a ser añadido a <i>expr</i>. Ejem: <code>SELECT ADDDATE('2021-01-02', 31);</code> → 2021-02-02
ADDTIME (<i>expr1</i> , <i>expr2</i>)	Añade <i>expr2</i> a <i>expr1</i> y devuelve el resultado. <i>Expr1</i> es una expresión en formato hora o fecha/hora y <i>expr2</i> es una expresión en formato hora. Ejem: <code>SELECT ADDTIME('2020-12-31 23:59:59.999999', '1 1:1:1.000002');</code> → 2021-01-02 01:01:01.000001 <code>SELECT ADDTIME('01:00:00.999999', '02:00:00.999998');</code> → 03:00:01.999997
PERIOD_ADD (<i>p</i> , <i>n</i>)	Añade <i>n</i> meses al período <i>p</i> (expresado en formato YYMM ó YYYYMM). Ejem: <code>SELECT PERIOD_ADD(202101,2);</code> → 202103
DATEDIFF (<i>expr1</i> , <i>expr2</i>)	Devuelve (<i>expr1-expr2</i>) expresado como un valor de días entre una fecha y otra. <i>Expr1</i> y <i>expr2</i> son expresiones de fecha o fecha/hora. Ejem: <code>SELECT DATEDIFF('2020-12-31 23:59:59','2020-12-30');</code> → 1 <code>SELECT DATEDIFF('2020-11-30 23:59:59','2020-12-31');</code> → -31
PERIOD_DIFF (<i>p1</i> , <i>p2</i>)	Devuelve el número de meses entre los períodos <i>p1</i> y <i>p2</i> (expresados en formato YYMM ó YYYYMM). Ejem: <code>SELECT PERIOD_DIFF(202002,202103);</code> → -13
DATE_FORMAT (<i>date</i> , <i>format</i> [, <i>locale</i>])	Formatea la fecha según el formato especificado. El lenguaje utilizado depende del valor de la variable del Sistema <i>lc_time_names</i>. Se pueden utilizar varios modificadores de <i>format</i>, entre los que se incluyen: - %a → Nombre corto del día de la semana. - %b → Nombre corto del mes. - %c → Mes con 1 ó 2 dígitos. - %d → Día con 2 dígitos. - %e → Día con 1 ó 2 dígitos. - %H → Hora con 2 dígitos entre 00-23. - %h → Hora con 2 dígitos entre 01-12. - %i → Minuto con 2 dígitos. - %j → Día del año (001-366). - %M → Nombre completo del mes. - %m → mes con 2 dígitos. - %r → Hora en formato de 12 horas seguido por AM/PM. - %S → Segundos con 2 dígitos. - %T → Hora en formato 24 horas. - %v → Número de semana (01-53) empezando en lunes. - %W → Nombre completo del día de la semana - %w → Día de la semana (0=domingo, 6=sábado). - %x → Año con 4 dígitos (primer día de la semana lunes). - %Y → Año con 4 dígitos. - %y → Año con 2 dígitos.

	Ejem: SELECT DATE_FORMAT('2021-02-16 22:23:00', '%W %M %Y'); → martes febrero 2021 SELECT DATE_FORMAT('2021-02-16 22:23:00', '%d %y %a %m %b %j'); → 16 21 mar 02 feb 047 SELECT DATE_FORMAT('1997-10-04 22:23:00', '%H %r %T %S %w'); → 22 10:23:00 PM 22:23:00 00 6
STR_TO_DATE (str, format)	Es la inversa de la función DATE_FORMAT(). Toma una cadena <i>str</i> y un formato de cadena <i>format</i> y devuelve un valor DATETIME si el formato de cadena contiene partes de fecha y hora, o un TIME si contiene solo horas, o un DATE si contiene solo fechas. Se utilizan modificadores iguales que el caso anterior. Ejem: SELECT STR_TO_DATE('Wednesday, June 2, 2014', '%W, %M %e, %Y'); → 2014-06-02
DAY (date) ó DAYOFMONTH (date)	Devuelve el día del mes de la fecha pasada. Ejem: SELECT DAYOFMONTH('2021-03-26'); → 26
DAYNAME (date)	Devuelve el día de la semana de la fecha pasada. Ejem: SELECT DAYNAME('2021-03-26'); → viernes
DAYOFWEEK (date) y WEEKDAY (date)	Devuelve el número del día de la semana (1=domingo... 7=sábado). Esto contrasta con la función WEEKDAY, que tiene índices diferentes (0=lunes... 6=domingo). Ejem: SELECT DAYOFWEEK('2021-03-26'); → 6 SELECT WEEKDAY('2021-03-26'); → 4
DAYOFYEAR (date)	Devuelve el día del año, entre 1 y 366. Ejem: SELECT DAYOFYEAR('2021-03-26'); → 85
HOURL (time) / MINUTE (time) / SECOND (time)	Extrae la hora, los minutos ó segundos de una hora dada. Ejem: SELECT HOUR(sysdate()); → 19
MONTHNAME (date)	Devuelve el nombre completo del mes de la fecha pasada Ejem: SELECT MONTHNAME('2021-02-03'); → febrero
YEAR (date)	Devuelve el año de una fecha dada. Ejem: SELECT YEAR('1987-01-01'); → 1987
LAST_DAY (date)	Obtiene el último día del mes al que pertenece la fecha. Devuelve un valor DATE Ejem: SELECT LAST_DAY(sysdate()); → 2021-01-31
EXTRACT (unit FROM date)	Extrae un valor numérico de una fecha concreta. El valor puede ser <i>day</i> (día), <i>month</i> (mes), <i>year</i> (año), etc. Ejem: SELECT EXTRACT (MONTH FROM SYSDATE()); → Devuelve 1. SELECT EXTRACT(YEAR_MONTH FROM '2020-07-02 01:02:03'); → 202007

6. Funciones de conversión

CONVERT y CAST

Sintaxis:

- CONVERT (expr, type)
- CAST (expr AS type)
- CONVERT (expr USING transcoding_name)

Las funciones CONVERT y CAST toman un valor de un tipo y lo convierten a otro.

La principal diferencia entre CONVERT y CAST es que CONVERT(expr, type) cumple con la sintaxis ODBC mientras que CAST (expr AS type) y CONVERT(... USING...) son sintaxis SQL92.

CONVERT con USING se utiliza para convertir datos entre diferentes conjuntos de caracteres (se puede consultar más información sobre conjuntos de caracteres en <https://mariadb.com/kb/en/character-set-and-collation-overview>). Por ejemplo, la instrucción `SELECT CONVERT('abc' USING utf8);` convierte la cadena 'abc' del conjunto de caracteres por defecto al conjunto de caracteres *utf8*.

Ejem:

```
SET @x = BINARY 'AardVark';
```

```
SELECT LOWER(@x), LOWER(CONVERT (@x USING latin1));
```

--Se convierte un dato binario a cadena para poder pasarlo a minúsculas

LOWER(@x)	LOWER(CONVERT (@x USING latin1))
BLOB	aardvark

Ejem:

```
SELECT CAST(2.345 AS DECIMAL(7,6)); → 2.345000
```

Para convertir una cadena a un valor numérico, normalmente no hay que hacer nada; sencillamente se usa el valor de la cadena como si se tratase de un número.

Ejem: `SELECT 1+'1';` → 2

De igual forma, si se usa un número en un contexto de cadena, el número se convertirá automáticamente a cadena *binary*.

Ejem: `SELECT CONCAT('hello you ',2);` → hello you 2

7. Funciones de control de flujo

OPERADOR CASE

Sintaxis:

```
CASE value WHEN [compare_value] THEN result [WHEN [compare_value] THEN result ...] [ELSE result] END
```

```
CASE WHEN [condition] THEN result [WHEN [condition] THEN result ...]
```

[ELSE result] END

La primera versión devuelve el resultado si *value=compare_value*. La segunda versión devuelve el resultado de la primera condición que sea verdadera. Si no hay ningún resultado coincidente, se devuelve el resultado especificado tras ELSE o NULL si no hay parte ELSE.

Ejem:

SELECT CASE 1 WHEN 1 THEN 'one' WHEN 2 THEN 'two' ELSE 'more' END; → one

SELECT CASE WHEN 1>0 THEN 'true' ELSE 'false' END; → true

FUNCIÓN IF

Sintaxis: IF(expr1, expr2, expr3);

Si *expr1* es verdadero (*expr1* <> 0 y *expr1* <> NULL) entonces la función devuelve *expr2*; si no, devuelve *expr3*. La función devuelve un valor numérico o cadena, dependiendo del contexto en el que se use.

Ejem: SELECT IF(1>2,2,3); → 3

*** EJERCICIOS – Realiza el ejercicio 4 ***

8. Funciones de listas

Las funciones de listas trabajan sobre un grupo de columnas dentro de una misma fila. Comparan los valores de varias columnas de la misma fila, para obtener el mayor o el menor. Su formato es:

GREATEST(valor1,valor2, ...) → Obtiene el valor mayor

LEAST(valor1,valor2, ...) → Obtiene el valor menor

Ejem: SELECT GREATEST(34.0,3.0,5.0,767.0); → 767.0

Ejem: Salario, comisión y valores mayor y menor de los cuatro de los empleados que no tienen la comisión nula:

SELECT salario, comision, GREATEST(salario, comision), LEAST(salario, comision) FROM emple WHERE comision IS NOT NULL;

salario	comision	GREATEST(salario, comision)	LEAST(salario, comision)
1500	390	1500	390
1625	650	1625	650
1600	1020	1600	1020
1350	0	1350	0

*** EJERCICIOS – Realiza el ejercicio 5 y 6 ***

9. Funciones de agrupación o de totales

Hasta ahora se han visto funciones que trabajan con valores simples, pero hay otras funciones estadísticas que actúan sobre un grupo de filas para obtener un valor, como, por ejemplo, el salario medio, el número de departamentos que hay en una ciudad, el empleado más antiguo, etc. Los valores nulos son ignorados y se realizan los cálculos sin contar con ellos.

Son muy interesantes en conjunción con agrupaciones (GROUP BY, que se verá en el siguiente tema) pues en estos casos ofrece muchas más posibilidades.

Función	Significado
COUNT (expresión) COUNT (DISTINCT expr, [expr...])	Cuenta las filas no nulas de la expresión. Se utiliza el asterisco para contar las filas. Puede anteponerse a la expresión DISTINCT, para que no cuente valores repetidos. Por defecto, cuanta los repetidos. Ejem: SELECT COUNT(*) FROM emple WHERE comision IS NOT NULL; → 4
SUM (expresión)	Suma los valores de la expresión Ejem: SELECT SUM(SALARIO) FROM emple WHERE comision IS NOT NULL; → 6075
AVG (expresión)	Calcula la media aritmética sobre la expresión indicada Ejem: SELECT AVG(SALARIO) FROM emple WHERE comision IS NOT NULL; → 1518.7500
MIN (expresión)	Mínimo valor que toma la expresión indicada Ejem: SELECT MIN(salario) FROM emple WHERE comision IS NOT NULL; → 1350
MAX (expresión)	Máximo valor que toma la expresión indicada SELECT MAX(salario) FROM emple WHERE comision IS NOT NULL; → 1625

*** EJERCICIOS – Realiza el ejercicio 7-16 ***

*** EJERCICIOS – Realiza el ejercicio 17-35 ***